Data Structure Algorithms Detailed Notes

Introduction

Algorithms are a set of instructions that are followed step by step to solve a problem. In Data Structures, algorithms are used to organize, search, sort, insert, delete, and manage data efficiently. A good algorithm saves time and memory. Different algorithms are useful for different types of problems. Some algorithms are simple and easy to understand, while some are more complex but provide faster performance.

In real life, algorithms are used everywhere. For example, when you search for a contact in your phone, sort files in a folder, find the shortest path in maps, or get suggestions on social media, algorithms are working in the background.

---

1. Searching Algorithms

Searching algorithms are used to find a specific element from a list, array, or any collection of data. Searching is very important because without it, it becomes difficult to locate the required information from large amounts of data.

Linear Search

Linear Search is the simplest searching algorithm. In this method, every element is checked one by one until the required value is found. If the element is found, its position is returned. If not found, the search continues until the end of the list.

Linear Search works on both sorted and unsorted data. It is easy to implement, but it becomes slow when the size of data becomes very large because every element may need to be checked.

For example, if an array contains 10, 20, 30, 40, and 50, and we want to find 40, then Linear Search will first check 10, then 20, then 30, and finally 40.

Time Complexity

- Best Case: O(1)
- Worst Case: O(n)

Binary Search

Binary Search is a faster searching algorithm, but it works only on sorted data. Instead of checking every element one by one, it divides the data into two halves again and again.

In Binary Search, first the middle element is checked. If the middle element is equal to the target value, then the search stops. If the target value is smaller than the middle element, then the search continues in the left half. If the target value is greater, then the search continues in the right half.

For example, in the sorted array 10, 20, 30, 40, 50, 60, 70, if we want to find 50, then first 40 is checked because it is in the middle. Since 50 is greater than 40, only the right half is searched.

Binary Search is much faster than Linear Search for large data because it reduces the search area again and again.

Time Complexity

- Best Case: O(1)
- Worst Case: O(log n)

---

2. Sorting Algorithms

Sorting algorithms are used to arrange data in ascending or descending order. Sorting is useful because organized data is easier to search, analyze, and manage.

Bubble Sort

Bubble Sort is one of the easiest sorting algorithms. In this method, adjacent elements are compared. If the first element is greater than the second element, then they are swapped.

This process is repeated again and again until the entire array becomes sorted. After each round, the largest element moves to the end of the array, just like a bubble moving upward in water.

For example, if the array is 5, 3, 8, and 1, then Bubble Sort will compare 5 and 3 and swap them. Then it compares 5 and 8, and after that 8 and 1. Multiple rounds are needed until the final sorted array becomes 1, 3, 5, and 8.

Bubble Sort is simple, but it is not suitable for large data because it takes more time.

Time Complexity

- Best Case: O(n)
- Worst Case: $O(n^2)$

Selection Sort

Selection Sort works by finding the smallest element from the unsorted part of the array and placing it at the correct position.

In the first step, the smallest element is found and swapped with the first element. In the second step, the second smallest element is found and swapped with the second position.

For example, in the array 29, 10, 14, and 37, the smallest value is 10, so it is placed at the first position.

Selection Sort is easy to understand, but it is also slow for large amounts of data.

Time Complexity

- Best Case: $O(n^2)$
- Worst Case: $O(n^2)$

Insertion Sort

Insertion Sort works similarly to arranging playing cards in your hand. One element is picked at a time and inserted into its correct position.

At first, the first element is considered sorted. Then the next element is compared with previous elements and placed in the correct location. This process continues until the entire array becomes sorted.

For example, if the array is 7, 4, 5, and 2, then 4 is inserted before 7, then 5 is inserted between 4 and 7.

Insertion Sort is useful for small data sets because it is simple and efficient for nearly sorted arrays.

Time Complexity

- Best Case: $O(n)$
- Worst Case: $O(n^2)$

Merge Sort

Merge Sort is a divide and conquer algorithm. It divides the array into smaller parts until only single elements remain. Then those smaller parts are merged back together in sorted order.

For example, if the array is 8, 3, 2, and 6, it is divided into two parts: 8 and 3, and 2 and 6. Then these parts are sorted and merged.

Merge Sort is very efficient for large data and provides consistent performance.

Time Complexity

- Best Case: O(n log n)
- Worst Case: O(n log n)

Quick Sort

Quick Sort is also a divide and conquer algorithm. It selects one element as a pivot and places smaller elements on one side and larger elements on the other side.

After partitioning, the same process is repeated on the smaller parts of the array.

Quick Sort is usually faster than many other sorting algorithms in practice and is widely used in programming.

Time Complexity

- Best Case: O(n log n)
- Worst Case: $O(n^2)$

---

3. Traversal Algorithms

Traversal means visiting every element of a data structure one by one.

Array Traversal

In array traversal, every element of the array is visited from the first position to the last position.

For example, if the array is 10, 20, 30, and 40, then the traversal output will be 10, 20, 30, and 40.

Linked List Traversal

In a linked list, traversal starts from the head node. Each node contains data and a link to the next node. The process continues until the next node becomes NULL.

Tree Traversal

Tree traversal means visiting every node of a tree.

Preorder Traversal

In Preorder Traversal, first the root node is visited, then the left child, and then the right child.

Inorder Traversal

In Inorder Traversal, first the left child is visited, then the root node, and then the right child.

Postorder Traversal

In Postorder Traversal, first the left child is visited, then the right child, and finally the root node.

---

4. Graph Algorithms

Graphs are used to represent relationships between objects. A graph contains nodes and edges. Nodes represent objects, and edges represent connections.

Breadth First Search (BFS)

BFS visits nodes level by level. It first visits all neighboring nodes and then moves to the next level.

BFS uses a queue data structure. It is useful for finding the shortest path in an unweighted graph.

For example, in a social media network, BFS can be used to find mutual friends.

Time Complexity

- O(V + E)

Depth First Search (DFS)

DFS visits nodes deeply before moving to another branch. It goes as far as possible in one direction and then comes back.

DFS uses stack or recursion. It is useful for solving maze problems, path finding, and cycle detection.

Time Complexity

- O(V + E)

---

5. Recursion

Recursion is a method in which a function calls itself again and again until a stopping condition is reached.

For example, factorial calculation is a common example of recursion.

Factorial of 5 means:
$5 \times 4 \times 3 \times 2 \times 1 = 120$

In recursive form:
Factorial(5) = 5 × Factorial(4)

Recursion is useful in tree traversal, searching, sorting, and mathematical calculations.

---

6. Divide and Conquer

Divide and Conquer is a problem-solving method in which a large problem is divided into smaller subproblems. Each smaller problem is solved separately, and then the results are combined.

Algorithms like Merge Sort, Quick Sort, and Binary Search use Divide and Conquer.

This method is useful because it makes difficult problems easier and faster to solve.

---

7. Greedy Algorithm

Greedy Algorithm always chooses the best possible option at the current step.

It does not think about the future result. It only focuses on the immediate best choice.

For example, if we want to make change for money using coins, the Greedy Algorithm will always choose the largest possible coin first.

Greedy algorithms are used in Dijkstra Algorithm, Huffman Coding, and Minimum Spanning Tree.

---

8. Dynamic Programming

Dynamic Programming is a method used to solve complex problems by breaking them into smaller parts.

In Dynamic Programming, the result of each smaller problem is stored so that it can be reused later. This avoids repeated calculations and saves time.

For example, in the Fibonacci series, the value of previous numbers is stored and reused.

Dynamic Programming is used in Knapsack Problem, Longest Common Subsequence, and shortest path problems.

---

9. RAG (Retrieval Augmented Generation)

RAG stands for Retrieval Augmented Generation. It is an AI technique in which a Large Language Model first retrieves information from documents and then generates an answer based on that information.

RAG is useful because a normal AI model only gives answers based on its training data. But in RAG, the model can also use external files such as PDFs, text files, Word documents, websites, or databases.

For example, if you create a chatbot for a company, then you can upload company PDFs and documents into the RAG system. When a user asks a question, the system searches the documents first and then gives an answer.

A basic RAG system contains the following steps:

1. Load the documents.
2. Split the documents into smaller chunks.
3. Convert chunks into embeddings.
4. Store embeddings into a vector database.
5. Search similar chunks when a user asks a question.
6. Send those chunks to the language model.
7. Generate the final answer.

---

10. Text Splitting in RAG

Text splitting is one of the most important parts of a RAG system. Large documents cannot be directly stored because they are too big. So documents are divided into smaller pieces called chunks.

If chunk size is too small, then important meaning may be lost. If chunk size is too large, then the model may not understand the content properly.

Character Text Splitter

Character Text Splitter divides text based on a fixed number of characters.

For example, if chunk size is 500 characters, then after every 500 characters a new chunk is created.

This method is simple, but sometimes it can break a sentence in the middle.

Recursive Character Text Splitter

Recursive Character Text Splitter is more advanced. It first tries to split text using paragraphs, then lines, then spaces, and finally characters.

This method is better because it tries to keep sentences meaningful.

For example, it first checks for

paragraph breaks. If the chunk is still too large, then it checks
line breaks. If still too large, then it checks spaces.

This is one of the most commonly used text splitters in RAG projects.

Token Text Splitter

Token Text Splitter divides text based on tokens instead of characters.

Tokens are smaller units of words used by language models.

For example, a sentence may contain 20 words but 30 tokens.

This method is useful because AI models work with tokens internally.

Sentence Splitter

Sentence Splitter breaks the text sentence by sentence.

This helps keep the meaning clear because complete sentences stay together.

---

11. Embeddings in RAG

Embeddings are numerical representations of text. They convert words, sentences, or paragraphs into vectors.

These vectors help the system understand similarity between two pieces of text.

For example, the sentence “Car is moving fast” and “Vehicle is running quickly” have different words, but their embeddings may be similar because the meaning is almost the same.

Embeddings are important because they help find the most relevant chunks during search.

Common embedding models are:

- OpenAI Embeddings
- Hugging Face Embeddings
- Sentence Transformers
- BGE Embeddings
- Instructor Embeddings

---

12. Vector Databases in RAG

A vector database is used to store embeddings. When the user asks a question, the query is converted into an embedding and compared with stored embeddings.

The most similar chunks are returned.

FAISS

FAISS stands for Facebook AI Similarity Search. It is one of the most popular vector databases.

FAISS is fast and useful for storing embeddings locally in a project.

It is commonly used in Python RAG systems.

ChromaDB

ChromaDB is another vector database used in RAG projects.

It is simple to use and works well with LangChain.

Pinecone

Pinecone is a cloud-based vector database.

It is useful when large amounts of embeddings need to be stored online.

Weaviate

Weaviate is also a vector database. It supports semantic search and machine learning features.

---

13. LangChain in RAG

LangChain is a Python framework used to build RAG applications.

It helps connect documents, text splitters, embeddings, vector databases, prompts, and language models.

LangChain makes it easier to build chatbots, PDF question answering systems, and document search systems.

Some important parts of LangChain are:

- Document Loaders
- Text Splitters
- Embeddings
- Vector Stores
- Retrievers
- Chains
- Memory
- Prompt Templates

For example, in a RAG project, LangChain can load a PDF, split it into chunks, create embeddings, store them in FAISS, and then generate answers.

---

14. Common Python Libraries Used in RAG

Many Python libraries are used while building a RAG system.

NumPy

NumPy is used for numerical calculations and array operations.

Pandas

Pandas is used for handling structured data like tables and CSV files.

Matplotlib

Matplotlib is used for graphs and visualizations.

Scikit-learn

Scikit-learn is used for machine learning tasks.

OpenAI

OpenAI library is used to access GPT models and embedding models.

LangChain

LangChain is used to connect all RAG components.

FAISS

FAISS library is used to store and search embeddings.

PyPDF

PyPDF is used to read text from PDF files.

PDFPlumber

PDFPlumber is another library used for extracting text from PDFs.

Docx2txt

Docx2txt is used to read Word documents.

BeautifulSoup

BeautifulSoup is used to extract data from HTML pages.

Requests

Requests library is used to send API calls and get website data.

Transformers

Transformers library from Hugging Face is used for models, tokenizers, and embeddings.

Sentence Transformers

Sentence Transformers library is used to create embeddings.

Tiktoken

Tiktoken is used for token counting.

Streamlit

Streamlit is used to create simple web applications for RAG systems.

Flask

Flask is used to create backend APIs and web applications.

FastAPI

FastAPI is also used to build APIs quickly.

---

15. Flow of a RAG System

The complete flow of a RAG system is:

1. User uploads documents.
2. Document loader reads the content.
3. Text splitter divides the content into chunks.

4. Embedding model converts chunks into vectors.
5. Vector database stores those vectors.
6. User asks a question.
7. The question is converted into an embedding.
8. Similar chunks are searched from the vector database.
9. Retrieved chunks are sent to the language model.
10. The model generates the final answer.

This process makes the answer more accurate and related to the uploaded documents.

Conclusion

Algorithms are very important in Data Structures because they help solve problems quickly and efficiently. Different algorithms are used for different tasks such as searching, sorting, traversal, graph processing, and optimization. Understanding these algorithms is important for programming, interviews, projects, and software development.